Name: Rachel Wilbanks

Date: October 23, 2026

Course: CS454

Assignment: CS454 Midterm

CS454 Midterm - OpenStack Deployment Analysis

1.) Resources and Choices

I would use a small Ubuntu VM in the instructor-provided OpenStack cloud. When it comes to size, I would use a flavor of 1-2 virtual CPUs and around 2 GB of RAM. Since Node.js deals with simple HTTP requests, the more modest VM size is suitable here. The image, router connection to an external network, flavor, and possibly the floating IP might already be provided by the cloud administrator. If not provided, they would need to be created and/or selected. Persistent block storage is not necessary here, as the request counter is stored in process memory and is made to reset whenever the application is restarted.

2.) Service Responsibilities

To start, Keystone authenticates and returns a scoped token tied to the project and tells the client where the other APIs currently live. Glance supplies the image image data that is used to build an instance. Nova gets the instance request and schedules a suitable host. Neutron looks over the instance's port on the network and subnet, applies the security group, provides the router and floating-IP connectivity. Once the VM exists, if not already, Docker Engine and Compose need to be installed. After installing (if it was not installed already), run "docker compose up" to start everything up. Docker is unaware of OpenStack, and mostly (if not solely) manages the container that is present. Personally, I would not use Cinder here.

3.) Request Path

The external client would send out a GET request to the VM's floating IP, then, when it reaches the router, it (the floating IP) is mapped to the VM's private IP. The security group must allow inbound traffic on port 8080 from the intended address(es). When the VM gets the request, Docker's port forwards it to port 8080 inside the container. The Node.js service processes the request and returns an HTTP response to the client. SSH access differs because it requires a

completely separate TCP 22 rule, which I would personally restrict to my own address. ICMP (or ping) can help, but is completely optional and not absolutely required to run the application.

4.) Failure Analysis

I would start by checking the health of the container, the application logs, and testing the process using the following: “docker compose ps”, “docker compose logs”, and “curl http://localhost:8080/health”. If there is an issue here, it is likely a container or application issue. Next, I would check to see if there is an issue with specific ports or if the client’s firewall could be blocking anything.

5.) Operational Complexity

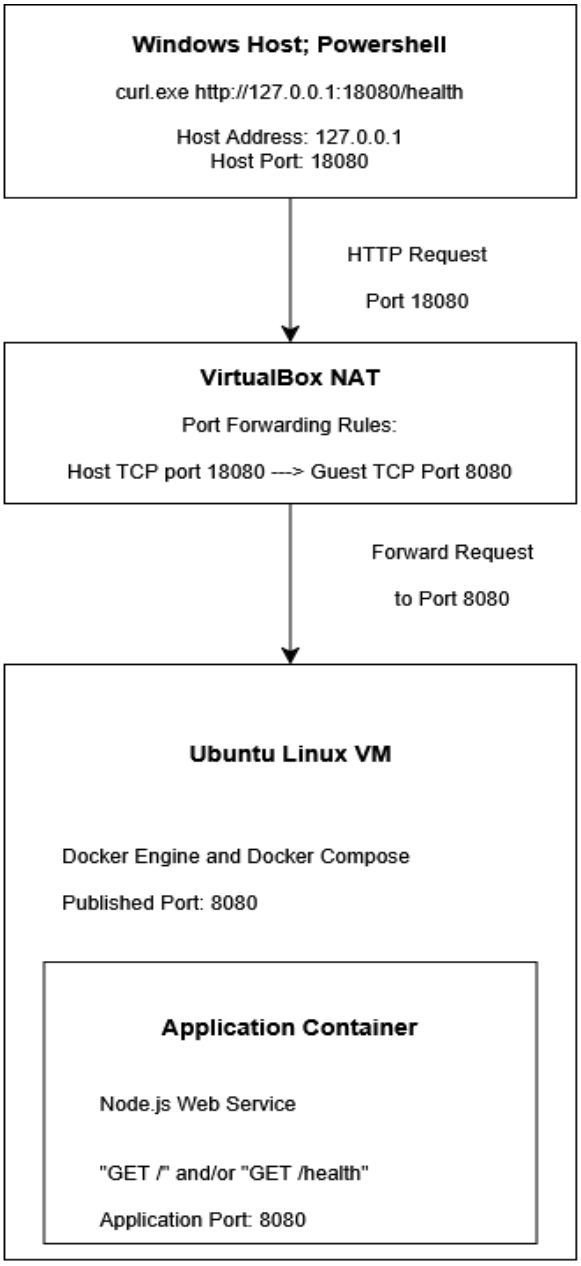
Operating an OpenStack cloud involves many more components and dependencies than deploying an application into an already working cloud since OpenStack operators have to manage infrastructure networking, services, storage, incident response, upgrades, etc., while, in an already working cloud, you can assume that most, if not all, of these are taken care of already.

6.) When A VM with Compose is Sufficient & Behavior Kubernetes Provided in the Experiment

A VM with Compose is satisfactory for a smaller service where brief downtime is ok and manual restarting is possible (in other words, a user needs to be able to restart if necessary).

In my experiment, Kubernetes was able to provide near, if not, automatic recovery of a pod when I deleted one. It did this without being asked or having to enter a command to do so, while Docker Compose restarted solely because I commanded it to. Using label selectors, Kubernetes also separated the pods from the service. That explains why “somethingWrong” (incorrect selector) broke connectivity in Part 4, even though the pods were still healthy.

7.) Diagram



Then the HTTP response will return to the Windows Host

Some Sources:

- CS454 Course Materials
- https://www.geeksforgeeks.org/cloud-computing/introduction-to-openstack/
- https://docs.docker.com/compose/intro/compose-application-model/
- https://www.openstack.org/software/project-navigator/openstack-components#openstack-services (Various pages)